

Aurora

Repositorio de GitHub: <https://github.com/sergiobelvisb/Aurora>

Autores:

Sergio Belvís Barba, Brian Camba Hipólito y Damiem Rave
Grizales

Fecha: 2025 - 2026

Nombre del Ciclo: Desarrollo de Aplicaciones Web

Curso escolar: 2º

Resumen

Se aconseja realizarlo una vez terminado el proyecto o en la fase final del mismo.

Aurora consiste en el desarrollo de una aplicación web orientada al ámbito médico, diseñada para la monitorización en tiempo real de la actividad cerebral de pacientes mediante dispositivos EEG (electroencefalográficos). El objetivo principal del sistema es proporcionar a médicos y profesionales sanitarios una herramienta digital que les permita observar, analizar y almacenar datos neurofisiológicos de los pacientes.

El sistema se apoya en una solución integral compuesta por hardware y software. En la parte hardware, se utilizan bandas Mindflex con sensores EEG conectadas a un Arduino Nano ESP32. Estas bandas son las encargadas de captar las señales cerebrales del paciente, procesarlas y enviarlas de manera inalámbrica mediante conexión Wi-Fi.

Los datos recogidos por el dispositivo son transmitidos en tiempo real a un servidor intermedio desarrollado en Node.js, el cual actúa como gestor de la comunicación mediante WebSockets. Este servidor es el encargado de mantener una conexión persistente entre los dispositivos físicos y la aplicación web, permitiendo así la transmisión continua de datos sin necesidad de recargas o peticiones constantes.

Por otro lado, la aplicación web principal ha sido desarrollada utilizando Laravel como framework backend. Este sistema se encarga de la gestión de usuarios, autenticación, almacenamiento de datos médicos, creación de perfiles de pacientes y administración general de la plataforma. Además, Laravel actúa como puente entre la base de datos y el resto de componentes del sistema, asegurando la persistencia y organización de la información.

El frontend de la aplicación está desarrollado en JavaScript puro, lo que permite una interacción dinámica con el usuario. Desde la interfaz, los médicos pueden iniciar la monitorización de un paciente mediante un botón que abre la conexión con el servidor WebSocket. A partir de ese momento, se reciben los datos en tiempo real que son representados visualmente mediante gráficas dinámicas que muestran la evolución de las ondas cerebrales.

Desde el punto de vista de la seguridad y la privacidad, Aurora ha sido diseñada teniendo en cuenta la sensibilidad de los datos médicos que maneja. El acceso a la plataforma está protegido mediante un sistema de autenticación robusto, y toda la información de los pacientes se almacena en una base de datos privada alojada en Amazon RDS, aislada de accesos externos mediante subredes privadas dentro de una VPC de AWS. De este modo, se garantiza la confidencialidad e integridad de los datos neurofisiológicos, cumpliendo con los principios básicos de protección de información en entornos sanitarios.

Este proyecto integra tecnologías modernas de desarrollo web y sistemas embebidos, combinando Laravel, Node.js, WebSockets, Arduino Nano ESP32 y JavaScript para crear una solución distribuida, escalable y orientada al tiempo real. Su principal valor reside en la capacidad de unir hardware biomédico con una plataforma web accesible, lo que facilita el trabajo clínico y mejora la capacidad de observación y análisis de la actividad cerebral de los pacientes.

Abstract

Traducción al inglés del apartado anterior

Aurora consists in the development of a web application oriented to the medical field, designed for the real-time monitoring of patients' brain activity by using EEG (electroencephalographic) devices. The main objective of the system is to provide doctors and healthcare professionals with a digital tool that allows them to observe, analyse and store neurophysiological data from patients.

The system is based on an integrated solution composed of both hardware and software components. On the hardware side, Mindflex headsets with EEG sensors connected to an Arduino Nano ESP32 are used. These headsets are responsible for capturing the patient's brain signals, performing initial processing and sending them wirelessly via Wi-Fi connection.

The data collected by the device is transmitted in real time to an intermediate server developed in Node.js, which acts as a communication manager by using WebSockets. This server is responsible for maintaining a persistent connection between the physical devices and the web application, enabling continuous data transmission without the need for page reloads or constant requests.

Furthermore, the main web application is developed by using Laravel as the backend framework. This system handles user management, authentication, medical data storage, patient profile creation and overall platform administration. Additionally, Laravel acts as a bridge between the database and the rest of the system components, ensuring data persistence and organization.

The frontend of the application is developed in pure JavaScript, by enabling dynamic interaction with the user. Through the interface, doctors can start monitoring a patient by pressing a button that opens a WebSocket connection with the server. From that moment on, real-time is received and visually represented through dynamic graphs that display the evolution of brain waves.

From a security and privacy perspective, Aurora has been designed with the sensitivity of the medical data it handles in mind. Access to the platform is protected by a robust authentication system, and all patient information is stored in a private database hosted on Amazon RDS, isolated from external access through private subnets within an AWS VPC. This ensures the confidentiality and integrity of neurophysiological data, in line with the fundamental principles of information protection in healthcare environments.

This project integrates modern web development technologies and embedded systems, combining Laravel, Node.js, WebSockets, Arduino Nano ESP32 and JavaScript to create a distributed, scalable and real-time-oriented solution. Its main value lies in the ability to connect biomedical hardware with an accessible web platform, facilitating clinical work and improving the ability to observe and analyse patients' brain activity

ÍNDICE

Introducción	6
Justificación	6
Objetivos	7
Planificación del Proyecto	9
Metodología de seguimiento y control	9
Reparto de Tareas y Diagrama de Gantt	9
Análisis de la ejecución	10
Desarrollo del proyecto	11
Dispositivo de medición	11
Base de datos	12
Backend	13
Comunicación en tiempo real mediante WebSockets	13
Frontend	14
Infraestructura en AWS	14
Pruebas del Sistema	15
Pruebas del dispositivo de medición (Arduino Nano ESP32)	15
Pruebas de comunicación en tiempo real.	15
Pruebas del backend	15
Pruebas del frontend	16
Pruebas de integración del sistema	16
Presupuesto	17
Inversión inicial.	17
Resumen de la inversión	19
Estudio del sector y viabilidad del proyecto	20
Factores tecnológicos	21
Factores legales	21
Factores económicos	21
Clientes potenciales	21
Competencia	21
Proveedores	21
Costes variables	23
Problemas encontrados y soluciones	24
- Desarrollo del proyecto prematuro:	24
- Planteamiento del trato de datos erróneo:	24
- Cambio de framework backend:	24
- Comunicación del análisis en tiempo real:	25
- Problemas de sincronización entre componentes del sistema:	25
- Problemas en las configuraciones de Laravel:	25
- Configuración automática de red WiFi en el dispositivo EEG.	26
Conclusiones	28
Futuras mejoras	29
Fuentes	30
Anexos	32

ÍNDICE DE FIGURAS

Figura 1. Diagrama de Gantt con la información cronológica del proyecto.	9
Figura 2. Fragmento del código del sistema automático de configuración WiFi.	11
Figura 3. Fragmento del código de la creación del punto de acceso WiFi.	12
Figura 4. Fragmento del código de la conexión de datos del Arduino con WebSocket.	13
Figura 5. Conexión al AP y visualización del formulario propio del Arduino.	32
Figura 6. Esquema de la arquitectura AWS.	33
Figura 7. Mensaje de error al descargarse una imagen oficial Docker.	33

ÍNDICE DE ANEXOS

ANEXO 1. Visuales de la conexión con el AP del Arduino y formulario web para añadir credenciales red.	32
ANEXO 2. Imagen de la arquitectura AWS.	33
ANEXO 3. Mensaje de error al descargarse una imagen oficial de Docker durante un clásico en LaLiga.	33

Introducción

Justificación

Las tecnologías EEG se utilizan actualmente en distintos sistemas de interacción cerebro-computadora y monitorización clínica (Zhang et al., 2020).

El proyecto Aurora es interesante debido a que combina los conocimientos de diseño, desarrollo y despliegue de aplicaciones web adquiridos a lo largo del curso, el uso de robótica y su aplicación en el sector de la salud.

La realización de este proyecto conlleva no solo el trabajo de saber programar si no que para llevarlo a cabo son necesarias numerosas horas de planificación para formarse en múltiples ámbitos como son la salud cerebral, la robótica y todo lo que supone desplegar la propia aplicación.

La salud mental es un tema igual de importante que la salud física y la forma en la que se realizan exámenes y pruebas para el diagnóstico de posibles enfermedades y trastornos es lo que nos ha llevado a realizar este proyecto. La idea general es sencilla, se miden las ondas cerebrales, se recogen esos datos y, se muestran de una forma gráfica entendible para especialistas. Todo esto con ayuda de nuestro dispositivo y la infraestructura de nuestra aplicación web.

Otro aspecto que hace interesante el proyecto es su capacidad de ampliación. La arquitectura planteada permite añadir nuevas funcionalidades en el futuro, como mejores sensores EEG, análisis avanzados de datos o sistemas de apoyo al diagnóstico mediante inteligencia artificial.

También consideramos importante el impacto educativo y profesional del proyecto, ya que nos ha permitido aplicar conocimientos reales en un entorno muy cercano al mundo laboral, al enfrentarnos a problemas técnicos, decisiones de arquitectura y planificación similares a los de un proyecto profesional.

Finalmente, Aurora destaca por tratarse de una idea innovadora dentro del contexto del curso, uniendo desarrollo web, sistemas en tiempo real y tecnología aplicada a la salud en un único proyecto con utilidad práctica y potencial de crecimiento.

El uso de dispositivos EEG ligeros y portátiles ha aumentado considerablemente en los últimos años gracias a la mejora de los sistemas de adquisición de señales cerebrales (Sciaraffa et al., 2022).

Objetivos

El objetivo principal del proyecto es desarrollar una aplicación web funcional y operativa capaz de gestionar pacientes y monitorizar en tiempo real datos obtenidos mediante un dispositivo de medición de ondas cerebrales.

Para alcanzar este objetivo principal, se han definido los siguientes objetivos específicos:

1 Desarrollo de la Aplicación Web.

1.1 Desarrollo de la lógica de la aplicación.

1.1.1 Base de datos

Diseñar, crear y probar una base de datos estructurada que permita almacenar de forma segura la información de pacientes, registros y mediciones realizadas.

1.1.2 Desarrollo Backend

Desarrollar la lógica interna de la aplicación mediante controladores y sistemas de gestión capaces de administrar usuarios, pacientes y comunicación de datos entre los distintos componentes del sistema.

1.1.3 Comunicación en tiempo real

Implementar un sistema de comunicación en tiempo real que permita visualizar las mediciones cerebrales instantáneamente desde la aplicación web.

1.2 Desarrollo de la interfaz gráfica.

1.2.1 Diseño estructural y visual

Diseñar una interfaz intuitiva, clara y accesible para facilitar la navegación y el uso de la aplicación por parte de los usuarios.

1.2.2 Visualización de datos

Representar gráficamente las mediciones obtenidas por el dispositivo para facilitar su interpretación por parte del personal médico.

2. Desarrollo del dispositivo de medición

2.1 Captura de ondas cerebrales

Desarrollar un dispositivo capaz de detectar y recoger datos relacionados con la actividad cerebral mediante sensores EEG.

2.2 Integración con la aplicación web

Conectar el dispositivo con la infraestructura de la aplicación para permitir el envío y recepción de datos en tiempo real.

3. Despliegue e infraestructura

3.1 Infraestructura cloud

Desplegar la aplicación y los servicios necesarios en la nube utilizando AWS para garantizar accesibilidad, estabilidad y escalabilidad.

3.2 Seguridad y estabilidad

Garantizar un funcionamiento estable y seguro tanto en el envío de datos como en la gestión de la información almacenada.

Planificación del Proyecto

Metodología de seguimiento y control

- **Gestión de tareas:** Se han utilizado herramientas de planificación para secuenciar las actividades en función de las necesidades de ejecución.
- **Documentación técnica:** Cada fase ha generado su propia documentación (scripts de base de datos, diagramas de arquitectura en AWS y documentación de la API en Laravel).
- **Control de versiones:** Se ha utilizado Git como herramienta colaborativa para el control del código fuente, asegurando la coherencia entre el frontend y el backend.
- **Diario de incidencias:** Se ha mantenido un registro de los problemas encontrados y sus soluciones (detallados en el apartado correspondiente de esta memoria) para facilitar la evaluación del proceso.

Reparto de Tareas y Diagrama de Gantt

Tareas del Proyecto		Responsable	H. Teóricas	H. Reales	Dic				Ene				Feb				Mar				Abr				May		
					S1	S2	S3	S4	S1	S2	S3	S4	S1	S2	S3	S4	S1	S2	S3	S4	S1	S2	S3	S4	S1	S2	S3
Fase 1: Análisis																											
	Definición de requisitos	Sergio, Damiem, Brian	1	2																							
	Investigación hardware	Sergio	1	2																							
	Diseño arquitectura AWS	Damiem	2	4																							
Fase 2: Diseño																											
	Mockups e interfaz	Sergio, Brian	2	4																							
	Modelo de Base de Datos	Brian	3	6																							
Fase 3: Hardware																											
	Programación firmware C++	Sergio	4	7																							
	Integración WebSockets ESP32	Sergio	4	8																							
Fase 4: Backend																											
	Estructura Laravel	Brian	5	12																							
	Gestión de Datos/API	Sergio	3	5																							
	Middleware de Seguridad	Brian	3	8																							
Fase 5: Frontend																											
	Maquetación HTML/CSS	Brian	2	4																							
	Lógica JS Vanilla	Sergio	3	6																							
	Gráficas en Tiempo Real	Sergio	3	5																							
Fase 6: DevOps																											
	Configuración EC2/VPC	Damiem	3	5																							
	Amazon RDS	Damiem	2	4																							
	Docker y Nginx	Damiem	2	3																							
Fase 7: Final																											
	Pruebas E2E	Damiem	3	5																							
	Redacción Memoria TFG	Sergio, Damiem, Brian	4	13																							
		TOTAL	50	103																							

Figura 1. Diagrama de Gantt con la información cronológica del proyecto.

El reparto de tareas entre los integrantes del equipo se ha definido en función de las áreas de conocimiento de cada miembro, aunque en determinadas fases del proyecto todos han colaborado de forma transversal para resolver problemas de integración y realizar pruebas conjuntas.

Integrante	Área principal	Tareas desarrolladas
Sergio	Hardware / WebSockets ESP32 / Gestión y tráfico de datos	Desarrollo del firmware en Arduino, integración del sensor EEG, realización e integración de WebSockets.
Brian	Base de datos / Backend / Frontend	Desarrollo de la aplicación web en Laravel, diseño del esquema de base de datos, gestión de autenticación y control de acceso.
Damiem	DevOps / Arquitectura / Despliegue	Desarrollo de infraestructura en AWS, despliegue con docker y configuración de VPC/ECS.

Análisis de la ejecución

Como se observa en el cronograma anterior, la planificación se ha ajustado a las fases cronológicas recomendadas por el equipo docente, destacando los siguientes puntos:

- **Solapamiento de tareas:** Durante los meses de marzo y abril existió una alta densidad de trabajo, ya que se realizaron de forma simultánea el desarrollo del backend en Laravel y la integración del frontend. Esta metodología permitió realizar pruebas de integración continuas y detectar incompatibilidades entre componentes de forma temprana, evitando que los errores se acumulasen en fases posteriores del proyecto.
- **Desviación de horas:** Las horas reales dedicadas a las fases de hardware (**Fase 3**) y backend (**Fase 4**) superaron la estimación teórica inicial. Esto se debió principalmente a la complejidad técnica que implica estabilizar la conexión vía WebSockets y a la migración de la lógica de datos hacia un entorno profesional en AWS para garantizar la escalabilidad del sistema. Aunque esta desviación supuso una carga de trabajo adicional, el equipo pudo absorber redistribuyendo tareas de menor prioridad hacia etapas más avanzadas del proyecto.
- **Hitos cumplidos:** Se logró finalizar la estructura de la base de datos y la maqueta estática antes de terminar el primer trimestre, cumpliendo así con los objetivos parciales establecidos en la temporalización oficial del proyecto. Este avance temprano proporcionó una base sólida sobre la que construir el resto de funcionalidades sin necesidad de replantear decisiones de arquitectura en fases avanzadas.
- **Gestión de recursos:** Para el desarrollo se han previsto y gestionado tanto recursos materiales, como los sensores MindFlex y los microcontroladores ESP32, como recursos de infraestructura cloud en AWS, asegurando siempre la viabilidad técnica del producto final. La selección de herramientas de software libre ha permitido minimizar los costes asociados al desarrollo, tal y como se detalla en el apartado de presupuesto.

Desarrollo del proyecto

A continuación se detalla el proceso de desarrollo seguido en el proyecto Aurora, estructurado en las distintas capas que componen la solución: el dispositivo de medición, el backend, la base de datos, el frontend y la infraestructura de despliegue en la nube.

Dispositivo de medición

La banda MindFlex captura la actividad cerebral mediante un sensor EEG y transmite los valores de las bandas de frecuencia (delta, theta, alfa, beta y gamma) por protocolo serie a una placa Arduino con módulo Wi-Fi. El firmware, desarrollado en C++ con la librería ThinkGear, procesa los datos y los envía en formato JSON mediante WebSocket directamente al servidor en AWS, eliminando cualquier equipo intermediario.

Las bandas de frecuencia recogidas representan distintos estados de actividad cerebral: las ondas delta están asociadas al sueño profundo, las theta a estados de relajación y somnolencia, las alfa a la calma consciente, las beta a la concentración activa y las gamma a procesos cognitivos de alta frecuencia.

La elección del Arduino Nano ESP32 como plataforma hardware se debe a su reducido tamaño, su bajo consumo energético y su capacidad de conectividad Wi-Fi integrada, lo que permite prescindir de cualquier dispositivo intermediario entre el sensor y el servidor. Todo el proceso de captura, procesamiento inicial y envío de datos se realiza de forma autónoma en el propio dispositivo.

Uno de los desarrollos más relevantes realizados en el dispositivo EEG fue la implementación de un sistema automático de configuración de red WiFi sobre el Arduino Nano ESP32. Esta funcionalidad permite configurar la conexión del dispositivo sin necesidad de modificar el código fuente ni volver a programar manualmente el microcontrolador.

```
void startAP() {  
  WiFi.softAP(AP_SSID, AP_PASS);  
  server.on("/", HTTP_GET, handleRoot);  
  server.on("/save", HTTP_POST, handleSave);  
  server.begin();  
}
```

Figura 2. Fragmento del código del sistema automático de configuración WiFi.

Este fragmento crea un punto de acceso WiFi propio utilizando el ESP32 mediante la función 'WiFi.softAP()'. Cuando el dispositivo no dispone de una red válida configurada o no consigue conectarse correctamente, entra automáticamente en modo Access Point (AP).

En este modo, el Arduino genera una red WiFi propia a la que el usuario puede conectarse desde un ordenador o dispositivo móvil. Además, el sistema inicia un pequeño servidor web interno que permite acceder a una página de configuración de las credenciales de la red WiFi deseada mediante un formulario HTML sencillo e intuitivo

Una vez recibidos los datos, el dispositivo almacena las credenciales utilizando el sistema de memoria persistente del ESP32:

```
String newSSID = server.arg("ssid");  
String newPass = server.arg("password");  
  
prefs.begin("wifi", false);  
prefs.putString("ssid", newSSID);  
prefs.putString("password", newPass);  
prefs.end();
```

Figura 3. Fragmento del código de la creación del punto de acceso WiFi.

Las credenciales quedan guardadas incluso después de apagar el dispositivo, permitiendo que el sistema se conecte automáticamente en futuros arranques sin necesidad de volver a configurarlo.

Una de las principales dificultades del proyecto ha sido el desarrollo del firmware en el Arduino Nano ESP32, especialmente en lo relacionado con la gestión de conectividad WiFi, almacenamiento persistente y comunicación en tiempo real. Se han trabajado con funcionalidades avanzadas del dispositivo como la creación de un punto de acceso, el uso de memoria no volátil mediante Preferences y la implementación de un sistema de configuración web embebido.

La conexión con la red WiFi y la página web desplegada desde el Arduino puede consultarse en el Anexo 1.

Base de datos

Se utiliza MySQL alojado en Amazon RDS dentro de una subred privada. El esquema relacional contempla cuatro entidades principales: Usuarios (especialistas y administradores), Pacientes, Sesiones y Mediciones, donde cada medición almacena los valores de cada banda de frecuencia junto con los indicadores de atención y meditación y su marca temporal.

La entidad Usuarios gestiona el acceso a la plataforma, diferenciándose entre administradores con control total del sistema y especialistas médicos con acceso restringido a sus propios pacientes. La entidad Pacientes recoge los datos personales y clínicos básicos de cada paciente registrado. Las sesiones agrupan las mediciones realizadas en una misma monitorización, lo que permite identificar el inicio y fin de cada prueba. Finalmente, las Mediciones almacenan cada lectura individual enviada por el dispositivo, incluyendo todos los valores de frecuencia y su marca temporal exacta.

Este diseño relacional permite consultar el historial completo de un paciente, comparar sesiones distintas y filtrar mediciones por rango de fechas, sentando las bases para futuras funcionalidades de análisis y generación de informes clínicos.

Se ha elegido MySQL debido a la naturaleza relacional de los datos del sistema. El proyecto incluye entidades claramente relacionadas como usuarios, pacientes, sesiones y mediciones, lo que encaja mejor con un modelo relacional que garantiza integridad referencial mediante claves foráneas.

Backend

El backend está desarrollado en PHP con el framework Laravel, exponiendo una API REST que permite la comunicación estructurada con la base de datos mediante peticiones HTTP.

Laravel ha sido elegido por su arquitectura MVC bien definida, su sistema de migraciones para la gestión de la base de datos y su robusto sistema de autenticación. La API REST expuesta por el backend sigue una estructura clara de rutas y controladores, separando la lógica de negocio de la capa de presentación y facilitando el mantenimiento y la ampliación futura del sistema.

La autenticación se gestiona mediante tokens, garantizando que únicamente los usuarios registrados puedan acceder a los recursos de la plataforma. Cada endpoint de la API valida los permisos del usuario antes de devolver o modificar cualquier dato, asegurando un control de acceso coherente en toda la aplicación.

Laravel ha sido seleccionado frente a otras alternativas debido a su arquitectura MVC, su sistema de autenticación integrado y su sistema de migraciones, lo que permite una mayor rapidez de desarrollo y una estructura más organizada.

Comunicación en tiempo real mediante WebSockets

Uno de los componentes más importantes del sistema es la comunicación en tiempo real entre el dispositivo EEG y la aplicación web. Para ello se desarrolló un servidor WebSocket en Node.js encargado de recibir las mediciones enviadas por el Arduino ESP32 y transmitir las a los clientes conectados.

```
wss.on('connection', (ws) => {
  console.log("Cliente conectado");

  clients.push(ws);

  ws.on('message', (message) => {
    console.log("Mensaje recibido:", message.toString());

    try {
      const datos = JSON.parse(message.toString());

      enviarAlApache(datos);

      clients.forEach(client => {
        if (client.readyState === WebSocket.OPEN) {
          client.send(JSON.stringify(datos));
        }
      });
    } catch (e) {
      console.error("JSON inválido:", e.message);
    }
  });

  ws.on('close', () => {
    console.log("Cliente desconectado");
    clients = clients.filter(c => c !== ws);
  });
});
```

Figura 4. Fragmento del código de la conexión de datos del Arduino con WebSocket.

Este fragmento de código implementa la lógica principal del sistema de comunicación en tiempo real. El servidor escucha conexiones entrantes en el puerto 8080 y mantiene una conexión persistente con cada cliente conectado.

Cuando el Arduino Nano ESP32 envía una medición EEG en formato JSON, el servidor procesa el mensaje y lo retransmite automáticamente al resto de clientes conectados, permitiendo visualizar las gráficas cerebrales en tiempo real desde la interfaz web.

Se ha elegido WebSockets sobre HTTP tradicional debido a la necesidad de mantener una comunicación continua y bidireccional entre el dispositivo y la aplicación web. Algunas alternativas se han descartado debido a la pérdida de control sobre la infraestructura y el flujo de datos.

La implementación de WebSockets en Node.js ha supuesto otro de los mayores retos técnicos del proyecto ya que no se ha tratado en el curso académico. Se ha tenido que aprender el funcionamiento de conexiones persistentes, gestión de múltiples clientes y envío de datos en tiempo real sin pérdida de rendimiento.

Frontend

La interfaz se ha construido con HTML, CSS, Bootstrap, JavaScript vanilla y el plugin de Javascript, chart.js para hacer las gráficas. La comunicación con el backend se realiza mediante la Fetch API de forma asíncrona. Las ondas cerebrales se representan en tiempo real con gráficas dinámicas con chart.js, lo que permite al especialista interpretar los datos sin necesidad de recargar la página.

El diseño de la interfaz ha priorizado la claridad y la usabilidad, dado que los usuarios finales son profesionales médicos que necesitan interpretar información de forma rápida y precisa. Los elementos visuales se han organizado de manera intuitiva, mostrando los datos del paciente, el estado de la conexión y las gráficas de actividad cerebral en una misma vista.

La conexión con el servidor WebSocket se establece desde el frontend mediante un botón de inicio de monitorización, a partir del cual los datos comienzan a llegar de forma continua y se representan gráficamente en tiempo real. El uso de JavaScript vanilla sin frameworks adicionales ha permitido mantener el control total sobre el comportamiento de la interfaz y reducir dependencias externas innecesarias.

Se ha optado por JavaScript vanilla en lugar de frameworks debido a la necesidad de un control directo del DOM y de las actualizaciones en tiempo real de las gráficas EEG.

Infraestructura en AWS

El despliegue se realiza sobre una VPC de AWS con cuatro subredes, un NAT Gateway y un Internet Gateway :

Subredes públicas: un balanceador de carga de aplicación (ALB) asociado a las dos subredes públicas sirviendo como punto de entrada del tráfico.

Subredes privadas: un cluster ECS ejecutando dos servicios cada uno con su contenedor desplegados mediante imágenes Docker personalizadas. Uno alojando la aplicación en Laravel y otro sirviendo el WebSocket para la comunicación con el Aurora Headset. Por otra parte, Amazon RDS con MySQL. Toda interconexión de servicios está previamente filtrada por los grupos de seguridad que ofrece Amazon Web Services.

Se ha seleccionado Amazon Web Service frente a soluciones como hosting local o servicios más limitados debido a su escalabilidad, seguridad y control de infraestructura. AWS permite segmentar el sistema en subredes públicas y privadas, lo que mejora la seguridad en el manejo de datos médicos.

El despliegue en AWS ha requerido aprendizaje en servicios cloud como ECS, RDS y configuración de redes privadas e implementación de conceptos de infraestructura y DevOps avanzados.

Imagen del esquema de la arquitectura en AWS se puede consultar en el Anexo 2.

Pruebas del Sistema

Durante el desarrollo de Aurora se han realizado diferentes pruebas con el objetivo de validar el correcto funcionamiento de cada uno de los componentes del sistema de forma individual y conjunta.

Pruebas del dispositivo de medición (Arduino Nano ESP32)

Se han realizado pruebas de:

- Conexión WiFi en diferentes redes.
- Funcionamiento del modo Access Point (AP)
- Almacenamiento y recuperación de credenciales WiFi.
- Lectura de datos EEG en tiempo real.
- Envío de datos mediante WebSocket.

Estas pruebas se han validado mediante monitor serie y verificación de recepción de datos en el servidor Node.js

Pruebas de comunicación en tiempo real.

Se han realizado pruebas específicas sobre el sistema WebSocket para comprobar:

- Estabilidad de la conexión persistente.
- Recepción continua de datos sin pérdida de paquetes.
- Envío simultáneo de datos a múltiples clientes.
- Latencia entre el dispositivo y la interfaz web.

Los resultados obtenidos han mostrado una transmisión estable en condiciones de red local.

Pruebas del backend

Se han realizado pruebas manuales sobre los endpoints de la API REST utilizando herramientas como Postman, verificando:

- Autenticación de usuarios mediante tokens.
- Creación y consulta de pacientes.
- Registro de sesiones y mediciones.
- Control de permisos de acceso.

Estas pruebas han permitido asegurar la correcta integración entre la base de datos y la lógica del sistema.

Pruebas del frontend

Se ha validado el correcto funcionamiento de la interfaz mediante:

- Visualización correcta de datos en tiempo real.
- Actualización dinámica de gráficas EEG con node.js.
- Interacción con el botón de inicio de monitorización.
- Comportamiento correcto ante desconexión del servidor.

Pruebas de integración del sistema

Finalmente, se han realizado pruebas de integración completa del sistema, verificando:

- Flujo completo desde el dispositivo hasta la base de datos.
- Transmisión de datos en tiempo real hasta la interfaz web.
- Persistencia de datos en sesiones de usuarios.
- Funcionamiento en entorno desplegado en AWS.

Presupuesto

1. Inversión inicial.

Para la puesta en marcha del proyecto se requiere una inversión inicial compuesta por los recursos necesarios tanto a nivel hardware, software e infraestructura tecnológica, así como el coste asociado al desarrollo del sistema.

1.1 Inversión en hardware

El proyecto requiere dispositivos físicos para la captura de datos biométricos mediante señales EEG.

Concepto	Unidades	Precio Unitario (€)	Total (€)
Arduino Nano ESP32	2	20	40
Banda EEG / Sensores	2	120	240
Material electrónico auxiliar	1	20	20
Total hardware			300

Se ha estimado el uso de dos dispositivos para permitir pruebas simultáneas y redundancia.

1.2 Inversión en software

El desarrollo se basa principalmente en herramientas de software libre, lo que reduce significativamente los costes.

Concepto	Coste (€)
Entorno de desarrollo (VS Code, Node.js , Laravel)	0
Sistema de control de versiones	0
Total software	0

Aunque no suponen coste económico, son herramientas clave para el desarrollo del proyecto.

1.3 Infraestructura en la nube (AWS)

El sistema se despliega en la nube utilizando servicios de AWS, lo que permite escalabilidad y alta disponibilidad.

Servicio	Coste Mensual (€)	Coste Anual (€)
Servidor web (EC2)	15	180
Servidor WebSocket (EC2)	15	180
Base de datos (RDS)	20	240
Almacenamiento y tráfico	10	120
Total infraestructura	60/mes	720/año

Se ha estimado un uso básico-medio adecuado para una fase inicial del proyecto.

1.4 Coste de desarrollo

El coste de desarrollo representa el valor real del trabajo realizado, calculado según el número de horas dedicadas.

Concepto	Horas	Precio/hora (€)	Total (€)
Desarrollo backend	80	20	1600
Desarrollo tiempo real (Node.js + WebSocket)	60	20	1200
Desarrollo frontend	60	20	1200
Integración hardware	50	20	1000
Pruebas y documentación	50	20	1000
Total desarrollo	300		6000

Se ha utilizado una tarifa de desarrollador junior (20€/h), acorde al nivel formativo del proyecto.

2. Resumen de la inversión

Concepto	Coste (€)
Hardware	300
Software	0
Infraestructura	720
Desarrollo	6000
Total inversión	7020

La mayor parte de la inversión corresponde al desarrollo del sistema, ya que el proyecto requiere programación backend, frontend, comunicación en tiempo real e integración con dispositivos EEG.

Los costes de infraestructura se han calculado estimando un año completo de funcionamiento en AWS, incluyendo servidores, bases de datos y almacenamiento.

Por otro lado, el coste de hardware es reducido debido al uso de componentes electrónicos accesibles y reutilizables.

3. Financiación del proyecto

La inversión inicial del proyecto asciende a 7020 €. Para cubrir este importe se plantea una financiación combinada entre aportación propia y ayudas públicas.

Fuente de financiación	Importe (€)	Porcentaje
Aportación propia de los integrantes	2020	28,8%
Subvención pública para innovación tecnológica	3000	42,7%
Préstamo bancario para emprendedores	2000	28,5%
Total financiación	7020	100%

3.1 Aportación propia

Los integrantes del proyecto realizarán una aportación inicial de 2020 €, destinada principalmente a cubrir la compra del hardware, los gastos iniciales de infraestructura cloud y el material electrónico auxiliar.

3.2 Subvención pública

Se contempla la posibilidad de solicitar una subvención orientada a proyectos de innovación tecnológica y digitalización sanitaria por un importe estimado de 3000 €.

3.3 Préstamo bancario

Para completar la financiación se plantea un préstamo bancario de 2000 € destinado a cubrir parte de los costes de desarrollo y mantenimiento inicial del sistema.

3.4 Viabilidad económica

El proyecto presenta una viabilidad económica favorable, ya que la inversión inicial es moderada para un proyecto sanitario-tecnológico, gran parte del software utilizado es gratuito y la infraestructura cloud permite ajustar costes según la demanda.

4. Estudio del sector y viabilidad del proyecto

4.1 Caracterización del sector

Aurora se sitúa dentro del sector tecnológico-sanitario, concretamente en el ámbito de la monitorización biomédica y las aplicaciones web orientadas a la salud. Este sector se encuentra en crecimiento debido al aumento de la digitalización de los servicios sanitarios y a la necesidad de herramientas capaces de almacenar y visualizar información médica en tiempo real.

La demanda de soluciones de monitorización y análisis de datos biomédicos ha aumentado durante los últimos años debido al desarrollo de tecnologías IoT, la expansión de la telemedicina y la creciente inversión en innovación sanitaria. Además, el uso de dispositivos conectados para la recogida de información médica permite optimizar procesos clínicos y mejorar el seguimiento de pacientes.

Desde el punto de vista tecnológico, el proyecto se beneficia del crecimiento de infraestructuras cloud y tecnologías de comunicación en tiempo real como WebSockets, las cuales facilitan el desarrollo de aplicaciones escalables y accesibles desde cualquier ubicación.

4.2 Macroentorno

Factores tecnológicos

El proyecto se desarrolla en un entorno tecnológico en constante evolución, donde herramientas como AWS, Docker, Node.js o Laravel permiten construir sistemas distribuidos de forma más eficiente y escalable. Además, el crecimiento de dispositivos IoT y sensores biomédicos facilita la creación de soluciones de monitorización médica conectadas en tiempo real.

Factores legales

Al tratarse de un sistema orientado al ámbito sanitario, el proyecto debe considerar normativas relacionadas con la protección de datos personales y médicos, especialmente el Reglamento General de Protección de Datos (RGPD) de la Unión Europea. Por este motivo, la información de los pacientes se almacena en entornos privados y protegidos dentro de AWS.

Factores económicos

El uso de herramientas open source reduce considerablemente los costes de desarrollo e implementación. Además, la infraestructura cloud permite adaptar el gasto económico según el crecimiento del sistema y la cantidad de usuarios activos.

4.3 Microentorno

Clientes potenciales

El proyecto está orientado principalmente a clínicas, centros médicos, consultas privadas y entornos de investigación relacionados con neurociencia y monitorización cerebral.

Los usuarios directos de la plataforma serían profesionales sanitarios que necesiten visualizar y almacenar información EEG de pacientes de forma centralizada y accesible.

Competencia

Existen empresas especializadas en sistemas EEG profesionales como NeuroSky, OpenBCI o Bitbrain, las cuales ofrecen soluciones avanzadas de monitorización cerebral. Sin embargo, muchas de estas plataformas utilizan hardware y software propietarios con costes elevados.

Aurora plantea una alternativa basada en tecnologías accesibles y componentes reutilizables, enfocándose principalmente en la integración entre dispositivos EEG, aplicaciones web y comunicación en tiempo real.

Proveedores

Los principales proveedores del proyecto corresponden al hardware electrónico y servicios cloud. Entre ellos destacan AWS para la infraestructura en la nube y distribuidores de componentes electrónicos como sensores EEG y placas ESP32.

4.4 Análisis DAFO.

Fortalezas

Integración de múltiples tecnologías modernas

Arquitectura escalable y distribuida

Comunicación en tiempo real mediante WebSockets

Bajo coste respecto a soluciones comerciales

Debilidades

Uso de hardware EEG no profesional

Recursos económicos limitados

Complejidad técnica elevada

Proyecto en fase prototipo

Oportunidades

Crecimiento del sector health-tech

Aumento de la digitalización sanitaria

Posible integración futura de IA

Expansión de sistemas IoT médicos

Amenazas

Competencia de empresas especializadas

Cambios en normativas sanitarias

Rápida evolución tecnológica

Costes futuros de infraestructura

4.5 Clasificación de costes.

Costes fijos

Los costes fijos son aquellos que se mantienen independientemente del número de usuarios o sesiones realizadas.

Concepto	Coste mensual (€)
Infraestructura AWS	60
Conexión a internet	30
Equipos informáticos	40
Dominios y servicios auxiliares	10
Total costes fijos	140/mes

Costes variables

Los costes variables dependen del uso y crecimiento del sistema.

Concepto	Coste aproximado
Situación de hardware EEG	Variable
Incremento de tráfico y almacenamiento cloud	Variable
Mantenimiento y aplicación cloud	Variable

4.6 Previsión de ingresos

Aunque Aurora se ha desarrollado como proyecto académico y prototipo funcional, se ha realizado una estimación hipotética de ingresos en caso de comercialización del sistema como servicio SaaS orientado a clínicas y centros médicos.

Servicio	Precio estimado (€)
Suscripción mensual básica	80/mes
Suscripción profesional	150/mes
Instalación y configuración inicial	300

4.7 Ayudas y subvenciones

Para la puesta en marcha de un proyecto de estas características podrían solicitarse distintas ayudas públicas relacionadas con innovación tecnológica, digitalización y emprendimiento.

Entre las posibles líneas de ayuda destacan:

- Programas de ayuda del [Portal PYME](#)
- Ayudas para digitalización y emprendimiento de la [Comunidad de Madrid](#)
- Líneas de financiación ICO para emprendedores mediante [ICO Empresas y Emprendedores](#)
- Programas de asesoramiento y apoyo de la [Cámara de Comercio de Madrid](#)

Estas ayudas permitirían reducir la inversión inicial y facilitar la viabilidad económica del proyecto en sus primeras fases de desarrollo.

Problemas encontrados y soluciones

- Desarrollo del proyecto prematuro:

Problema:

Comenzamos a trabajar con el proyecto demasiado rápido y de forma poco eficiente sin saberlo. A medida que avanzábamos en el curso nos dimos cuenta de ello y tuvimos que adaptar el trabajo ya hecho a un estilo mejor.

Solución:

Se realizó una reorganización de la estructura del proyecto, adaptando el código y la arquitectura a metodologías más actuales y mantenibles. Además, se revisó la planificación del desarrollo para trabajar de forma más ordenada y escalable, aplicando mejores prácticas aprendidas durante el curso.

- Planteamiento del trato de datos erróneo:

Problema:

Al planear el despliegue de la arquitectura nos dimos cuenta de que el paso de los datos de medición del dispositivo Aurora al servidor era más complejo de lo necesario. Ya que la infraestructura se despliega en AWS, es más sencillo enviar los datos recogidos en un JSON por HTTPS en lugar de pasarlos a un ordenador por Bluetooth y después al servidor (en AWS).

Solución:

Se rediseñó el flujo de envío de datos para simplificar la arquitectura. En lugar de depender de un ordenador intermedio mediante Bluetooth, el dispositivo envía directamente la información al servidor desplegado en AWS utilizando peticiones HTTPS con formato JSON, lo cual mejora la eficiencia, estabilidad y escalabilidad del sistema.

- Cambio de framework backend:

Problema:

Tuvimos que cambiar el framework backend utilizado en el proyecto. Inicialmente optamos por utilizar uno utilizado en clase, lo que servía para el estudio, pero nos limitaba las funcionalidades, escalabilidad y buenas prácticas para el trabajo profesional.

Solución:

Se migró el backend completo a Laravel, reorganizando la arquitectura del proyecto y adaptando las funcionalidades ya desarrolladas al nuevo framework. Aunque supuso rehacer parte del trabajo, permitió implementar una estructura más profesional, segura y preparada para futuras ampliaciones.

- **Comunicación del análisis en tiempo real:**

Problema:

La implementación de la comunicación en tiempo real entre el dispositivo y la web supuso un reto inicial al integrar Node.js con WebSockets. La gestión de conexiones, envío de datos y sincronización entre cliente y servidor requirió múltiples pruebas y reajustes para poder hacer la conexión en tiempo real.

Solución:

Se realizaron distintas pruebas de conexión y optimización hasta conseguir una comunicación estable mediante WebSockets. Además, se separó la lógica de tiempo real en un servidor específico con Node.js, lo cual facilitó la gestión de conexiones simultáneas y mejoró la estabilidad del envío de datos en tiempo real.

- **Problemas de sincronización entre componentes del sistema:**

Problema:

Al trabajar con múltiples tecnologías (Laravel, Node.js, C++, JS), surgieron problemas estructurales y lógicos para coordinar correctamente el flujo de datos entre todos los componentes.

Solución:

Se definió una estructura de comunicación clara entre los distintos componentes del sistema, estableciendo formatos de datos comunes y separando responsabilidades entre backend, servidor en tiempo real y dispositivo físico. Esto permitió mejorar la sincronización y reducir errores en el intercambio de información.

- **Problemas en las configuraciones de Laravel:**

Problema:

A la hora de hacer pruebas para migrar la aplicación de local a AWS surgieron distintos problemas debido a versiones y dependencias necesarias para que Laravel funcionara correctamente.

Solución:

Se revisaron y ajustaron las versiones de PHP, Composer y las dependencias necesarias para Laravel, adaptando la configuración del entorno AWS a los requisitos del framework. También se realizaron pruebas de despliegue y configuración para garantizar el correcto funcionamiento de la aplicación tanto en local como en producción.

- Configuración automática de red WiFi en el dispositivo EEG.

Problema:

Durante el desarrollo del sistema surgió un problema relacionado con la conectividad del dispositivo basado en Arduino Nano ESP32. El dispositivo necesitaba disponer de una conexión WiFi válida para poder enviar los datos EEG al servidor mediante WebSockets. Sin embargo, en muchas ocasiones el dispositivo se iniciaba sin una red configurada o con credenciales incorrectas, impidiendo completamente la comunicación con la aplicación web y dificultando la configuración por parte del usuario.

Además, al tratarse de un dispositivo orientado a un entorno médico y portátil, no resultaba práctico modificar manualmente el código del Arduino cada vez que se quisiera conectar a una nueva red WiFi.

Solución:

Se desarrolló un sistema de configuración automática de red integrado directamente en el Arduino Nano ESP32. Cuando el dispositivo no detecta credenciales almacenadas o no consigue conectarse a la red configurada, el sistema crea automáticamente un punto de acceso WiFi propio.

A través de este punto de acceso, el usuario puede conectarse desde cualquier dispositivo móvil u ordenador y acceder a una página web de configuración alojada directamente en el propio Arduino. Esta interfaz web permite introducir el nombre y contraseña de la red WiFi deseada mediante un formulario sencillo e intuitivo.

Una vez introducidas las credenciales, estas se almacenan en la memoria interna del dispositivo utilizando el sistema Preferences de ESP32. Posteriormente, el dispositivo se reinicia automáticamente e intenta conectarse a la nueva red configurada.

- Configuración de variables de entorno.

Problema:

Durante el despliegue de la aplicación en laravel nos dio problema a la hora de enviar los formularios de login y registros por la configuración de las variables de entorno del fichero .env ya que no las teníamos adaptadas a la configuración que requieren al utilizar https.

Solución:

Se revisó la configuración de las variables de entorno utilizadas por Laravel y se detectó que el entorno desplegado en AWS no estaba utilizando correctamente los valores definidos en el fichero .env. Para solucionarlo, se añadieron manualmente las variables necesarias dentro de la configuración del entorno en AWS, incluyendo parámetros como APP_URL utilizando el protocolo https://, así como las credenciales y configuraciones necesarias para las sesiones y autenticación.

- **Problemas de acceso a imágenes oficiales de Docker.**

Problema:

Durante el despliegue del servidor Node.js encargado de la comunicación mediante WebSockets, surgió un problema inesperado al intentar descargar imágenes oficiales de Docker desde Docker Hub. Temporalmente, algunas IP relacionadas con servicios de Docker quedaron inaccesibles debido a bloqueos aplicados por medidas anti piratería de LaLiga durante la celebración de un partido entre el FC Barcelona y el Real Madrid. Esto impedía descargar correctamente las imágenes necesarias para continuar con el despliegue y las pruebas del sistema.

Solución:

Para poder continuar con el desarrollo y realizar pruebas temporales del entorno, se optó por utilizar una imagen alternativa no oficial compatible con Node.js. Una vez finalizó el bloqueo y volvió a estar disponible el acceso normal a Docker Hub, se sustituyó dicha imagen por la versión oficial correspondiente, manteniendo así una configuración más segura, estable y alineada con las buenas prácticas de despliegue.

Mensaje de error corroborando el problema con LaLiga e imágenes oficiales de Docker puede ser visitado en el Anexo 3.

Conclusiones

El desarrollo de AURORA ha permitido cumplir los objetivos planteados al inicio. Se ha desarrollado un sistema que permite la gestión de pacientes y la monitorización de datos en tiempo real dentro de un entorno web gracias a una banda con sensores electroencefalográficos que envían sus datos en tiempo real a través de un Arduino Nano ESP32.

A lo largo del proceso, se ha comprobado que no todas las soluciones son válidas para cualquier contexto, y que la elección de herramientas como Laravel o Node.js depende directamente de los requisitos del proyecto, como la necesidad de gestionar datos en tiempo real o estructurar correctamente la lógica de proyecto.

Por otro lado, el hecho de empezar un proyecto único y original nos ha permitido investigar todas las funcionalidades que nos hayan podido servir y comparar con el proyecto si podría ser una buena implementación. Esto ha añadido dificultad y ralentización en el trabajo pero con la recompensa de crear algo innovador y útil para el diagnóstico médico.

Además, el desarrollo del proyecto nos ha permitido adquirir conocimientos prácticos en distintas áreas como desarrollo web, arquitectura cloud, comunicación en tiempo real y tratamiento de datos. También hemos aprendido la importancia de una buena planificación y de mantener una arquitectura escalable y mantenible desde las primeras fases del proyecto.

Aurora no solo ha supuesto un reto técnico, sino también una experiencia de aprendizaje muy completa, ya que nos ha obligado a investigar tecnologías y soluciones fuera del contenido visto en clase, enfrentándonos a problemas reales similares a los que pueden surgir en un entorno profesional.

Futuras mejoras

A pesar de que el sistema desarrollado cumple con los objetivos planteados inicialmente, existen múltiples líneas de mejora y ampliación que permitirían aumentar tanto la precisión del sistema como sus funcionalidades y aplicabilidad en entornos reales.

Una de las mejoras más importantes estaría relacionada con el hardware utilizado para la captación de señales cerebrales. Actualmente, el proyecto utiliza una banda Mindflex con un sensor EEG básico conectado a un Arduino Nano ESP32. Aunque este dispositivo resulta adecuado para el desarrollo de prototipos y pruebas funcionales, presenta limitaciones en cuanto a precisión, cantidad de datos obtenidos y calidad de señales captadas.

Como futura mejora, se plantea la utilización de bandas EEG profesionales o cascos completos con múltiples sensores distribuidos en diferentes zonas del cráneo, como el del proyecto Mindtooth. Este tipo de dispositivos permitiría obtener lecturas mucho más precisas y detalladas de la actividad cerebral, lo que aumentaría considerablemente la fiabilidad de los datos recogidos. Además, el uso de múltiples canales EEG facilitaría la monitorización simultánea de distintas regiones cerebrales, permitiendo realizar análisis más avanzados y obtener información clínica de mayor valor.

El uso de dispositivos EEG multicanal permite obtener mediciones más precisas y detalladas de la actividad cerebral (BitBrain, 2024).

Otra posible mejora consistiría en la incorporación de algoritmos de inteligencia artificial y aprendizaje automático capaces de analizar los patrones cerebrales recogidos por el sistema. De este modo, la aplicación podría ofrecer predicciones, detección de anomalías o prediagnósticos más avanzados, lo cual ayudaría al personal médico en la toma de decisiones.

También se contempla la posibilidad de ampliar la plataforma con un sistema de almacenamiento histórico y generación de informes médicos automáticos, permitiendo comparar la evolución de un paciente a lo largo del tiempo mediante gráficas, estadísticas y registros de sesiones anteriores.

Fuentes

Bibliografía

Las siguientes fuentes han sido utilizadas tanto para la investigación y contextualización del proyecto como para el aprendizaje de tecnologías, estudio de señales EEG y consulta de referencias técnicas relacionadas con el desarrollo de Aurora.

Mika E. (21 febrero 2010) *How to Hack Toy EEGs*. FrontierNerds.

<https://frontiernerds.com/brain-hack>

Zhang Y, Shane X, Wang H, Zhang Z. (18 agosto 2020) *Data Analytics in Steady-State Visual Evoked Potential-Based Brain-Computer Interface: A Review*. IEEE Xplore.

<https://ieeexplore.ieee.org/abstract/document/9170622>

Gembler F, Stawicki P, Rezeika A, Benda M, Volosyak I. (14 diciembre 2020) *Exploring Session-to-Session Transfer for Brain-Computer Interfaces based on Code-Modulated Visual Evoked Potentials*. IEEE Xplore.

<https://ieeexplore.ieee.org/abstract/document/9170622>

Zhang Y, Quan S, Li Z, Zhao Y, Qian K, Zhang Z. (27-30 septiembre 2022) *CCA-based Spatio-temporal Filtering for Enhancing SSVEP Detection*. Universities of Leeds, Sheffield and York.

https://eprints.whiterose.ac.uk/id/eprint/195717/1/CCA_based_Spatio_temporal_Filtering_for_Enhancing_SSVEP_Detection.pdf

Spüler M, Rosenstiel W, Bogdan M. (7 diciembre 2012) *Online Adaptation of a c-VEP Brain-Computer Interface(BCI) Based on Error-Related Potentials and Unsupervised Learning*. PLOS ONE.

<https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0051077>

O'Reilly C, Larson E, Richards J, Elsabbagh M. (20 junio 2020) *Structural templates for imaging EEG cortical sources in infants*. bioRxiv.

<https://www.biorxiv.org/content/10.1101/2020.06.20.162131v1.full>

Sciaraffa N, Di Flumeri G, Germano D, Giorgi A, Di Florio A, Borghini G, Vozzi A, Ronca V, Babiloni F, Aricò P. (14 julio 2022) *Evaluation of a New Lightweight EEG Technology for Translational Applications of Passive Brain-Computer Interfaces*. Frontiers.

<https://www.frontiersin.org/journals/human-neuroscience/articles/10.3389/fnhum.2022.901387/full>

EEG Electrode Placement: Fixed vs. Variable. BitBrain. (26 marzo 2024)

<https://www.bitbrain.com/blog/eeg-electrode-placement>

Types of EEG Electrodes: Gel, Water and Dry. TMSi. (24 enero 2025)

<https://www.tmsi.artinis.com/blog/types-of-eeg-electrodes>

Johnag. (2013) *Mini Arduino Portable EEG - Brain Wave Monitor* +. Instructables.

<https://www.instructables.com/Mini-Arduino-Portable-EEG-Brain-Wave-Monitor-/>

Mika E. (1 marzo 2010) *brain-grapher*. GitHub. <https://github.com/kitschpatrol/brain-grapher>

Schlegel A. (30 julio 2015) *controlP5*. Sojamo. <https://www.sojamo.de/libraries/controlP5/>

Amazon Web Services. (2026). *Amazon Web Services Documentation*.

https://docs.aws.amazon.com/?nc2=h_uts_docs&trk=ea4f1bda-c329-47b3-a6d0-278045ca3aea&sc_channel=ps

Arduino (2026). *Arduino Documentation*.

https://docs.arduino.cc/?_gl=1*1n4mh4f*_up*MQ..*_ga*MTE4NTI1MDcxMi4xNzc5MDM4NTM4*_ga_NEXN8H46L5*czE3NzkwMzg1MzckbzEkZzAkDE3NzkwMzg1MzckajYwJGwwJGgyMDAxODUzMzQw

Laravel. (2026) *Laravel - The clean stack for Artisans and webs*.

<https://laravel.com/docs/13.x>

Node.js (2026). *Index | Node js v26.1.0 Documentation*. <https://nodejs.org/docs/latest/api/>

WebSockets (2026). *WebSockets - API web. | MDN*

https://developer.mozilla.org/es/docs/Web/API/WebSockets_API

Bootstrap (2026). *Bootstrap · The most popular HTML, CSS and JS library in the world*.

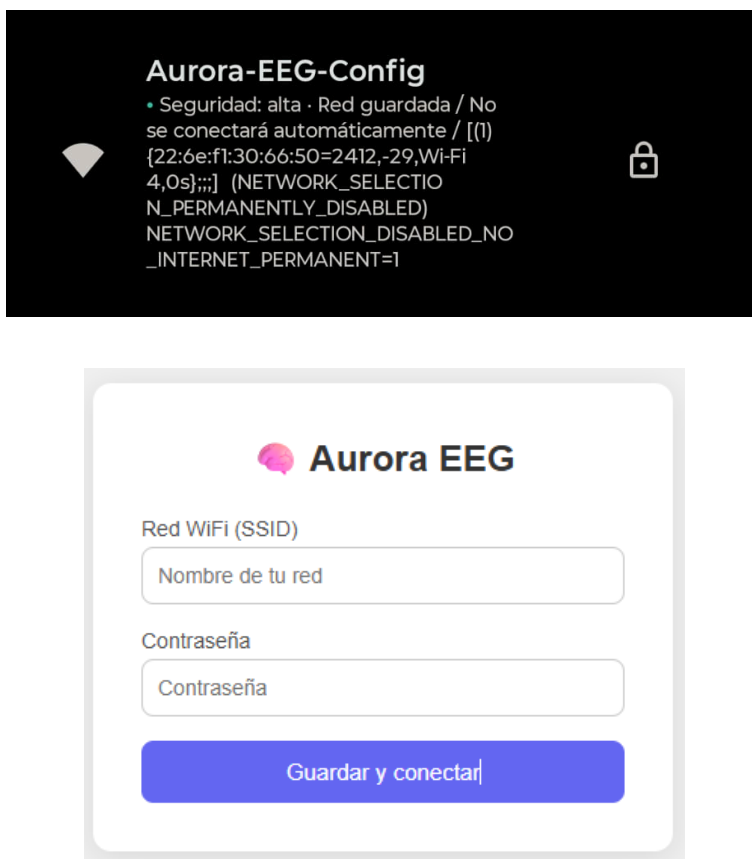
<https://getbootstrap.com/>

Chart.js (2026). *Chart.js | Open Source HTML5 chart for your website*.

<https://www.chartjs.org/>

Anexos

ANEXO 1. Visuales de la conexión con el AP del Arduino y formulario web para añadir credenciales red.



The image shows two parts of the Aurora-EEG interface. The top part is a dark-themed configuration screen titled 'Aurora-EEG-Config'. It displays network status: 'Seguridad: alta · Red guardada / No se conectará automáticamente / [(1)] {22:6e:f1:30:66:50=2412,-29,Wi-Fi 4,0s};;;] (NETWORK_SELECTIO N_PERMANENTLY_DISABLED) NETWORK_SELECTION_DISABLED_NO _INTERNET_PERMANENT=1'. There is a Wi-Fi icon on the left and a lock icon on the right. The bottom part is a light-themed web form titled 'Aurora EEG' with a brain icon. It contains two input fields: 'Red WiFi (SSID)' with the placeholder 'Nombre de tu red' and 'Contraseña' with the placeholder 'Contraseña'. Below these fields is a blue button labeled 'Guardar y conectar'.

Figura 5. Conexión al AP y visualización del formulario propio del Arduino.

ANEXO 2. Imagen de la arquitectura AWS.

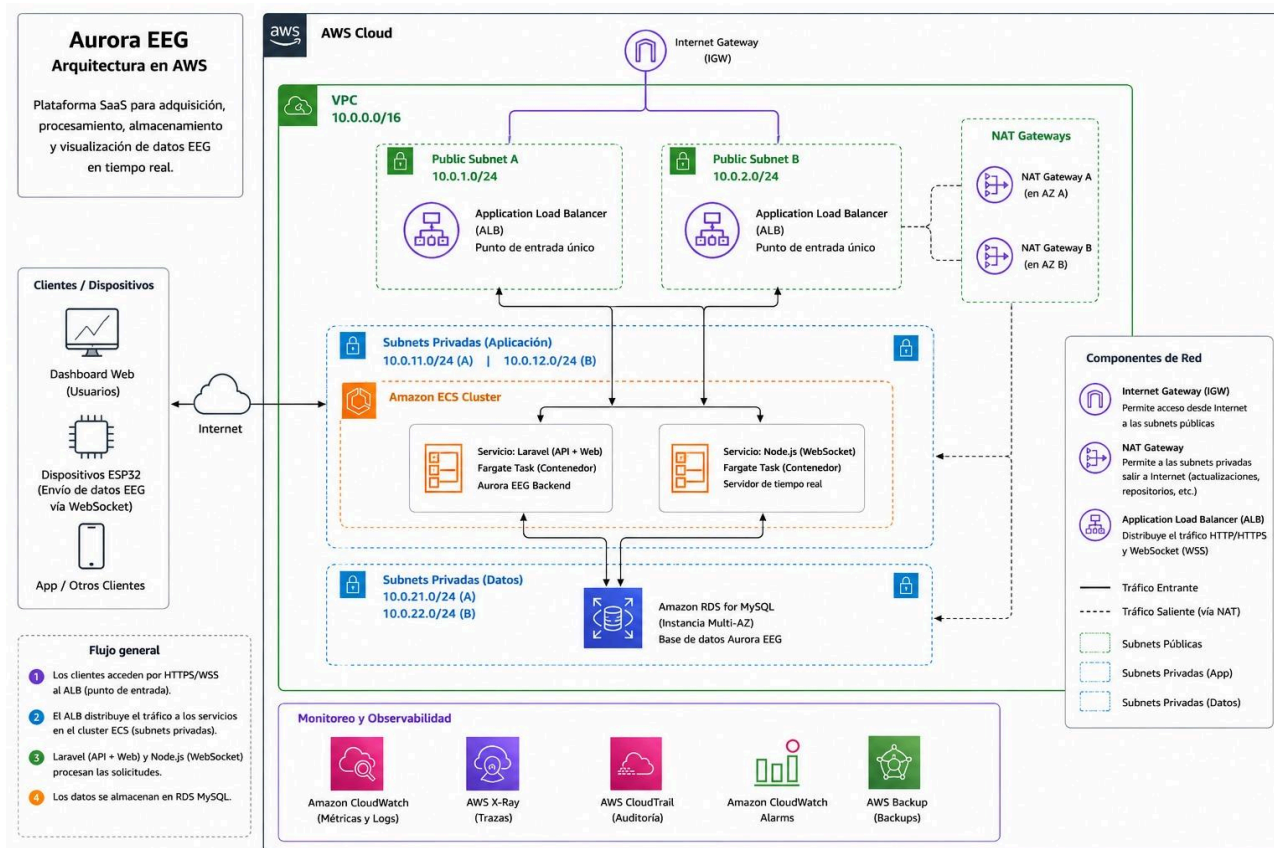


Figura 6. Esquema de la arquitectura AWS.

ANEXO 3. Mensaje de error al descargarse una imagen oficial de Docker durante un clásico en LaLiga.

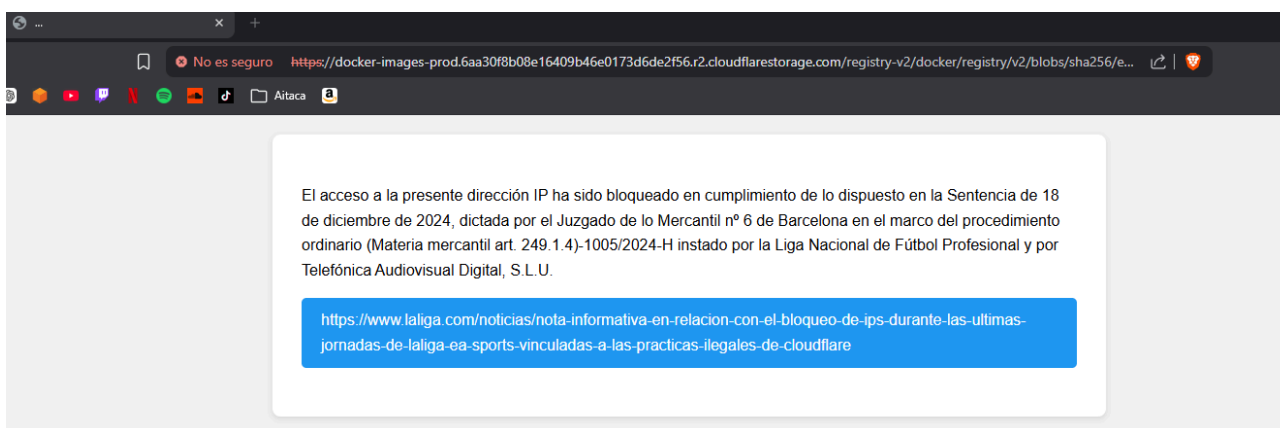


Figura 7. Mensaje de error al descargarse una imagen oficial Docker.